

The `search` function scans through the string, looks for the first match of the regex and returns the corresponding match object. It returns `None` in case the search fails. The syntax for search function is:

```
re.search(pattern, string, flags=0)
```

`pattern` is the regular expression, `string` is the target string on which we are performing the search operation, `flags` is an optional parameter which is used to modify the expression's behavior like performing a `case-insensitive` matching or a `multi-line` matching. Multiple flags can be specified using a bitwise `OR` operator.

```
>>> matches = re.search(r "my (.*?) is (.*?)", "my name is david");
>>> matches.group()
'my name is david'
```

`matches` is a `match` object returned by the search, it has a `True` value whenever there is a successful match and a `None` when a match fails. It also provides a `groups()` and a `group(n)` method which contains all the matched expressions. The value of `n` defaults to `0` if not supplied to the function.

Correspondingly, a `match` function scans the string and looks for a match of the regex which is anchored to the beginning of the string. It follows an identical syntax and return type as the search function.

```
re.match(pattern, string, flags=0)
```

`pattern`, `string`, `flags` perform the exact same function as in the search operation.

```
>>> match = re.match(r "my (.*?) is (.*?)", "Hi, my name is david");
>>> match.groups()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'NoneType' object has no attribute 'groups'
```

Even though the target string contains the regex pattern, we still don't get a successful match since the start of the string does not match the regex, hence the match returns a `None`.